

АГЕНТЫ ИИ / РАЗРАБОТКА ИИ / ЭКОСИСТЕМА CLOUD NATIVE

Почему агентские решения не работают в облачных системах

Агентам-кодировщикам нужна обратная связь для самокоррекции. В облачных системах сложнее всего обеспечить этот сигнал с той же скоростью и в том же масштабе, что и у агентов.

13 мая 2026 года, 9:00 [Арджун Айер](#)



Studio HD для Unsplash+

[Signadot](#) спонсировала этот пост.

Агенты по написанию кода работают эффективнее, если у них есть инструменты, рекомендации и обратная связь, которые помогают понять, что делать, как это делать и как узнать, работает ли это. В облачных системах обеспечить обратную связь не так просто.

Я несколько дней размышлял над **недавним постом Адди Османи о работе агентных систем**. Я согласен с одним из аргументов: агент для написания кода — это модель плюс все, что ее окружает (подсказки,

сильнее, чем обновление модели. Эдди указывает на данные Biga триведи, которые показывают, что одна и та же модель, установленная на другой стенд, поднялась с 30-го на 5-е место в рейтинге Terminal Bench 2.0. После того как вы увидите несколько таких скачков, вам будет сложно воспринимать выбор модели как интересный эксперимент.

Я хочу добавить один нюанс, который проявляется, когда задача агента — изменить код в облачной системе. Структура «обвязки» сохраняется, как и идея о том, что в основе ее работы лежат циклы обратной связи. Сложность заключается в том, что на самом деле означает «обратная связь» в распределенной системе и какая инфраструктура должна существовать, чтобы агент мог получать ее в рамках своего цикла.

Компоненты «оснащения» агента не равнозначны. Подсказки определяют, что именно пытается сделать агент. Такие инструменты, как MCP и интерфейсы командной строки, определяют, на что он может воздействовать. Навыки определяют, как он пытается внести изменения и какие инструменты использует. Политики и «песочницы» определяют ограничения для его действий. Все это влияет на то, что может делать агент и как он себя ведет. Ни один из этих компонентов не сообщает агенту, удалась ли попытка. Это задача сигнала обратной связи. Обратная связь — это то, что замыкает цикл, то, что превращает агента из генератора с разомкнутым контуром в нечто, способное самокорректироваться и завершать работу без вмешательства.

Для тех видов работы, о которых пишут Эдди и другие (одно приложение, запускаемое локально, с возможностью оценки через браузер), инфраструктура для такой обратной связи в основном проста. Для распределенных систем обеспечить такой же сигнал гораздо сложнее.



Signadot is a Kubernetes-native platform that empowers AI coding agents to verify code at scale. Combining fast, scalable ephemeral environments with a validation framework built for complex distributed systems, Signadot ensures high-velocity code generation results in safely merged pull requests.

[Learn More →](#)

Representation, Mirror, Okteto, and Signadot

18 June 2026

Watch Claude Code Validate a Microservices Change in a Closed Loop

4 June 2026

Signadot Plans

20 May 2026

HEAR MORE FROM OUR SPONSOR

svo1975pvn1982@gmail.com

Submit

What good feedback looks like in a harness

Addy uses a framing I find useful: success is silent, failures are verbose. If typecheck passes, the agent hears nothing. If it fails, the error gets injected into the loop and the agent self-corrects. The same shape holds for tests, lint, and any deterministic check. Hooks at lifecycle points (pre-commit, post-edit, pre-PR) separate “I told the agent to do X” from “the system enforces X.” Without enforced feedback, the agent’s other components are just suggestions.

Anthropic’s work on long-running harnesses pushes this further. Their three-agent setup (planner, generator, evaluator) drove a Playwright session that clicked through the running app and graded it against criteria. Across multi-hour runs, the result was working full-stack applications. The load-bearing piece was the evaluator’s feedback.

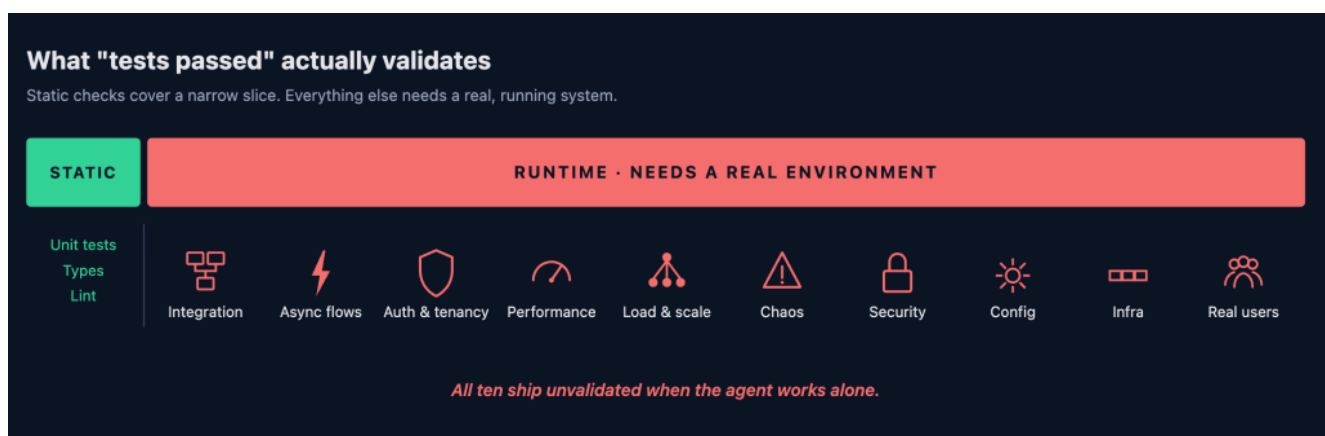
TRENDING STORIES

1. [Backporting bug fixes is dead, Project Valkey now sends in the bots](#)
2. ["An agent is an LLM and a harness": What Nvidia really thinks about OpenClaw](#)
3. [The Anthropic leader who built Claude Code says he ditched prompting — now he just writes loops.](#)

This is the picture for single-application work, runnable locally, where “does this work” can be answered by spinning up the dev server and clicking around. Most production software does not fit that shape.

What cloud native breaks

Production code in a **microservices architecture** lives inside a distributed system. A change in one service must be applied to the real services it depends on: the database, the message bus, the config, and the auth layer. The signal an agent needs to know its change works is the system’s actual response at runtime, and none of that exists inside a local sandbox. Tests that “pass” without that feedback miss most of the validation surface.



Three workarounds show up in practice, and all three break the feedback loop at the same point.

Mocks and stubs. Fine for unit-level work. But the bugs that matter in distributed systems emerge from interactions between real services with real state, and mocks reproduce neither of these.

CI after the PR opens. Real environments exist there, but the signal arrives outside the agent’s loop. The agent finishes, opens a PR, and waits. Failures at this stage are expensive: the agent has lost its working context, a human is now in the loop, and cycle time has stretched from minutes to hours. The feedback exists, but it comes too late and too inefficiently.

generate ten times the branch volume, and ten times the previews (each a full system clone) is not a budget conversation anyone wants to have.

“In cloud-native systems, the standard harness model gets the agent to “I think this works” and then stops.”

In cloud-native systems, the standard harness model gets the agent to “I think this works” and then stops. The feedback that would tell it otherwise either does not exist, arrives too late, or does not scale. The agent has no choice but to hand off to a human to validate the behavior and close the loop. The feedback component that the harness depends on needs better infrastructure to function as intended in a distributed system.

The role of environments in a complete harness

If the feedback that matters in cloud-native systems is runtime response, the harness needs a runtime to point the agent to. Not a sandbox, which isolates the agent’s execution from the host, but an environment, which gives the agent a real, runnable, system-level surface to validate against. The sandbox protects the host from the agent. The environment provides the feedback the agent needs to validate whether its change works in the real system and correct it if it doesn’t. Both belong in the harness, and they do different jobs.

“The sandbox protects the host from the agent. The environment provides the feedback the agent needs to validate whether its change works in the real system and correct it if it doesn’t.”

Four properties must be met for an environment to function as an effective harness at agent speed and scale.

Cheap and fast. Not a full staging clone. A lightweight ephemeral environment that shares stable infrastructure and isolates only what the agent’s change touches. If spinning one up costs as much as a full preview, the math fails. If it takes more than a few seconds, it falls outside the agent’s iteration window.

conditions that production exhibits.

Programmable. The agent has to spin one up, exercise it, read the results back, and tear it down. If a human has to provision or interpret, the loop is broken before it starts.

Isolated. The environment needs to provide realistic feedback in a single shared environment while remaining fully isolated from other changes to avoid collisions with other agents. One shared cluster needs to serve hundreds or thousands of agents working in parallel without contention.

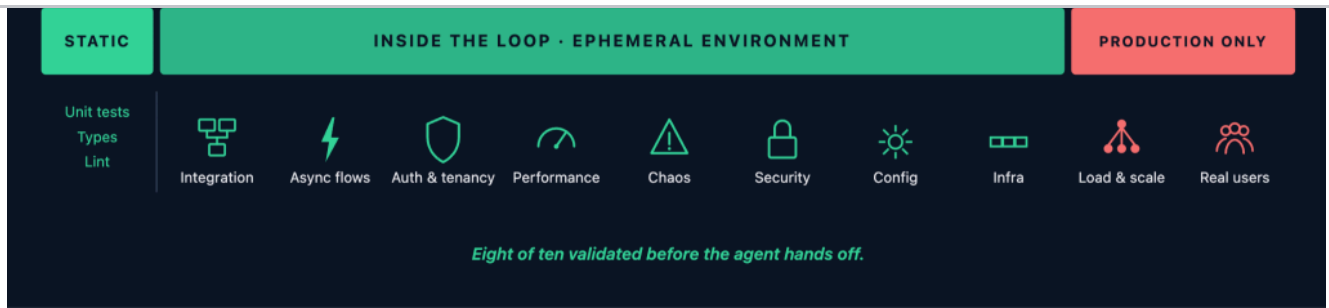
None of these requirements is exotic. **Platform teams have been building** toward them for years in the service of human developers. What **changes for agentic development** is treating them as part of the harness the agent itself drives, rather than infrastructure that sits beside the loop and gets consulted at PR time.

What this enables

Once the agent has runtime feedback inside its loop, the validation surface expands well beyond what local checks can cover. The agent can run integration paths end-to-end against the actual call graph between services. It can verify contract behavior against real consumers and producers, not generated stubs. It can hit the real database, observe real transaction behavior, and catch migrations that break unrelated queries.

These are the checks most engineers run by hand today, mostly after the agent is done, through some combination of local reproduction, staging, and PR-stage CI. None of them are tractable when the harness ends at a local sandbox, which is what limits how far an agent can autonomously pursue a change.

With runtime feedback, the agent does not just produce a diff that compiles and passes unit tests. It produces a change with verified evidence: traces showing the code ran, tests showing the integration holds, and observability data showing nothing else regressed. The diff arrives at PR review already validated against the parts of the system that matter. The reviewer's job shifts from "does this work" to "should we ship this," which is a different and more valuable kind of attention.



Closing the loop

Harness engineering has the right shape: coding agents need scaffolding around them to be useful, and feedback loops are central to that scaffolding. The components that shape an agent's first attempt have come a long way, and the typecheck-and-test feedback that closes the loop on local work is well understood. What is still missing for engineering teams building distributed systems is the infrastructure that enables the same kind of feedback against a real, running system at agent speed and scale.

That is what we **work on at Signadot**: lightweight environments that share a baseline infrastructure, isolate per change, and provide agents with a programmable runtime to validate against. Realistic feedback at agent speed and scale, inside the development loop.

TNS



Arjun Iyer, CEO of Signadot, is a seasoned expert in the cloud native realm with a deep passion for enhancing the developer experience. Boasting over 25 years of industry experience, Arjun has a rich history of developing internet-scale software and...

[Read more from Arjun Iyer →](#)

TNS owner Insight Partners is an investor in: Real, Anthropic.

SHARE THIS STORY



PENDING STORIES

... Backporting bug fixes is dead, Project Valkey now sends in the bots

he just writes loops.

4. Anthropic gives @Claude a permanent seat in your Slack channels

5. Claude Code vs. Cursor vs. Codex vs. Antigravity — six months in

INSIGHTS FROM OUR SPONSOR



Signadot is a Kubernetes-native platform that empowers AI coding agents to verify code at scale. Combining fast, scalable ephemeral environments with a validation framework built for complex distributed systems, Signadot ensures high-velocity code generation results in safely merged pull requests.

[Learn More →](#)

[Comparing Kubernetes Development Tools in 2026: Telepresence, mirrord, Okteto, and Signadot](#)

18 June 2026

[Watch Claude Code Validate a Microservices Change in a Closed Loop](#)

4 June 2026

[Signadot Plans](#)

20 May 2026

[AI Code Testing for Cloud-Native Systems: Why Unit Tests Aren't Enough](#)

8 May 2026

[Introducing the signadot-validate Skill: Close the Loop for Coding Agents Building Microservices](#)

8 May 2026

[Closing the Loop for Coding Agents in Cloud-Native Systems](#)

TNS DAILY NEWSLETTER

Receive a free roundup of the
most recent TNS articles in
your inbox each day.

svo1975pvn1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds



Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

FOLLOW TNS



[Disclosures](#)

[Terms of Use](#)

[Advertising Terms & Conditions](#)

[Privacy Policy](#)

[Cookie Policy](#)